Scalable Neural Optimization From Theory to Practice

Machester-Oxford Symposium on Advances in AI



Mingfei Sun

2 Nov 2026

The University of Manchester • mingfei.sun@manchester.ac.uk • https://mingfeisun.github.io

Plan for today

1. **The neural optimization problem**

   Formal statement: supervised and RL objectives; pre-requisites: forward/backward pass

2. **Classical approaches**

   Newton • Gauss–Newton • Steepest descent • Adam • Natural gradient

3. **Matrix optimizers: Muon, Shampoo and SOAP**

   Optimizers that treat weight matrices as matrices, not flattened vectors

The Optimization Problem

The machine learning puzzle

Three pieces to the puzzle:

1 Approximation

Does there *exist* a neural net in my model family that fits the training data?

2 Optimization

If it does exist, can I *find* it?

3 Generalization

Does it *work well* on *unseen* data?

Focus

This tutorial focuses mainly on the second question.

The optimization problem: formal statement

Supervised learning

Network $f(\boldsymbol{x}, \boldsymbol{\theta})$, error $\ell(\hat{\boldsymbol{y}}, \boldsymbol{y})$, data $\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}_{i=1}^{N}$:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(f(\boldsymbol{x}^{(i)}, \boldsymbol{\theta}),\, \boldsymbol{y}^{(i)}\big)$$

Example (log-likelihood): $\ell = -\log p_{\boldsymbol{\theta}}(\boldsymbol{y} \mid \boldsymbol{x})$, e.g. cross-entropy $-\log \operatorname{softmax}\big(f(\boldsymbol{x}, \boldsymbol{\theta})\big)_{\boldsymbol{y}}$.

The data are **fixed** and do not depend on $\boldsymbol{\theta}$.

Reinforcement learning (policy gradient)

Policy $\pi_{\boldsymbol{\theta}}(a \mid s)$; maximize $J(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}}\big[R(\tau)\big]$. The policy gradient is

$$\nabla J = \mathbb{E}_{(s,a) \sim \pi_{\boldsymbol{\theta}}}\big[\hat{A}(s,a)\, \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a \mid s)\big],$$

the gradient of an **advantage-weighted log-likelihood**:

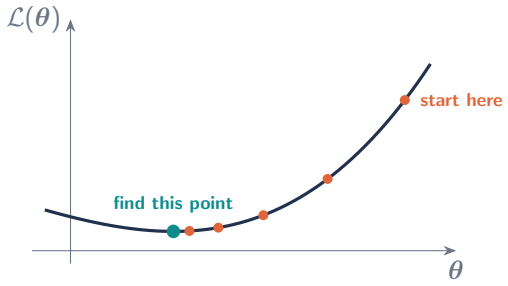
$$\mathcal{L}(\boldsymbol{\theta}) = -\,\mathbb{E}_{(s,a) \sim \pi_{\text{old}}}\big[\hat{A}(s,a)\, \log \pi_{\boldsymbol{\theta}}(a \mid s)\big]$$

Like supervised loss, with sampled actions as labels weighted by $\hat{A}$; but the data **depend** on the policy.

Goal

Find $\boldsymbol{\theta}$ that minimizes $\mathcal{L}(\boldsymbol{\theta})$: the same optimizers apply to both objectives.

The optimization problem: a picture



Roughly, we just iterate:

$$\boldsymbol{\theta} \longrightarrow \boldsymbol{\theta} - \underbrace{\eta}_{\text{learning rate}} \underbrace{\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}}_{\text{gradient}}$$

Computing the gradient: forward and backward passes



Forward pass

$h_0 = x$, and for layers $l = 1, \ldots, L$:

$$z_l = W_l\, h_{l-1} + b_l$$
$$h_l = \sigma(z_l) \quad \text{(element-wise)}$$

Output: $f(x; \theta) = h_L$
Parameters: $\theta = \{W_l, b_l\}_{l=1}^{L}$

Backward pass

Define $\delta_l = \frac{\partial \ell}{\partial z_l}$. Then

$$\delta_L = \frac{\partial \ell}{\partial h_L} \odot \sigma'(z_L)$$
$$\delta_l = \left(W_{l+1}^\top \delta_{l+1}\right) \odot \sigma'(z_l)$$
$$\frac{\partial \ell}{\partial W_l} = \delta_l\, h_{l-1}^\top, \qquad \frac{\partial \ell}{\partial b_l} = \delta_l$$

Per sample; the loss gradient $\frac{\partial \mathcal{L}}{\partial \theta}$ averages these over the training set. Note that $\frac{\partial \ell}{\partial W_l}$ is an **outer product** of two vectors, a fact we will reuse later.

$\odot$ denotes the element-wise (Hadamard) product: $(u \odot v)_i = u_i\, v_i$.

Classical 1st and 2nd Order Methods

Taylor expanding the loss

Starting point

Different classical approaches to optimization take this Taylor expansion as a starting point.

$$\mathcal{L}(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) = \underbrace{\mathcal{L}(\boldsymbol{\theta}) + \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}^{\top} \Delta\boldsymbol{\theta}}_{\text{“linearization”}} + \underbrace{\tfrac{1}{2}\Delta\boldsymbol{\theta}^{\top} \frac{\partial^2 \mathcal{L}}{\partial \boldsymbol{\theta}^2} \Delta\boldsymbol{\theta} + \cdots}_{\text{“non-linear part”}}$$

$$= \mathcal{L}(\boldsymbol{\theta}) + \boldsymbol{g}^{\top}\Delta\boldsymbol{\theta} + \tfrac{1}{2}\Delta\boldsymbol{\theta}^{\top}\boldsymbol{H}\,\Delta\boldsymbol{\theta} + \cdots$$

First-order methods

Use only the **linearization** $\mathcal{L}(\boldsymbol{\theta}) + \boldsymbol{g}^{\top}\Delta\boldsymbol{\theta}$, i.e. the gradient $\boldsymbol{g} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} \in \mathbb{R}^d$.

Second-order methods

Also use the quadratic term $\frac{1}{2}\Delta\boldsymbol{\theta}^{\top}\boldsymbol{H}\Delta\boldsymbol{\theta}$, i.e. the Hessian $\boldsymbol{H} = \frac{\partial^2 \mathcal{L}}{\partial \boldsymbol{\theta}^2} \in \mathbb{R}^{d \times d}$.

Second-order optimization: Newton's method

Take the Taylor expansion to second order and minimize over $\Delta\boldsymbol{\theta}$:

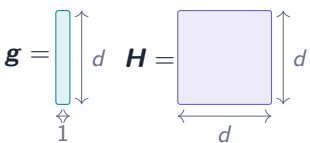
$$\mathcal{L}(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}) + \boldsymbol{g}^\top \Delta\boldsymbol{\theta} + \frac{\lambda}{2} \Delta\boldsymbol{\theta}^\top \boldsymbol{H} \Delta\boldsymbol{\theta} \qquad \Rightarrow \qquad \boldsymbol{g} + \lambda \boldsymbol{H} \Delta\boldsymbol{\theta} = 0$$

$$\Delta\boldsymbol{\theta} = -\frac{1}{\lambda} \boldsymbol{H}^{-1} \boldsymbol{g}$$ **Newton's method**

"pre-condition the gradient with the inverted Hessian"

Problems with Newton

- Might converge to a local max (fix: "cubic regularization").
- d parameters $\Rightarrow$ Hessian is $d \times d$: too expensive even for "small" networks.



Composite optimization: the GN decomposition

GN = **G**auss! **N**ewton!

Suppose we have a "composite" objective $\mathcal{L} = \ell \circ f$

e.g. error ℓ composed with neural net f

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} = \frac{\partial \ell}{\partial f} \cdot \frac{\partial f}{\partial \boldsymbol{\theta}}$$

chain rule

$$\frac{\partial^2 \mathcal{L}}{\partial \boldsymbol{\theta}^2} = \frac{\partial f}{\partial \boldsymbol{\theta}} \cdot \frac{\partial^2 \ell}{\partial f^2} \cdot \frac{\partial f}{\partial \boldsymbol{\theta}}^\top + \frac{\partial \ell}{\partial f} \frac{\partial^2 f}{\partial \boldsymbol{\theta}^2}$$

product rule + chain rule

Terminology

We call the second result the **Gauss–Newton decomposition** of the Hessian.

Composite optimization: the GN method

In the decomposition $\frac{\partial^2 \mathcal{L}}{\partial \boldsymbol{\theta}^2} = \frac{\partial f}{\partial \boldsymbol{\theta}} \frac{\partial^2 \ell}{\partial f^2} \frac{\partial f}{\partial \boldsymbol{\theta}}^\top + \frac{\partial \ell}{\partial f} \frac{\partial^2 f}{\partial \boldsymbol{\theta}^2}$, drop the second term, i.e. ignore the curvature of the model (!). Newton's method becomes

$$\Delta \boldsymbol{\theta} = -\left[\frac{\partial f}{\partial \boldsymbol{\theta}} \frac{\partial^2 \ell}{\partial f^2} \frac{\partial f}{\partial \boldsymbol{\theta}}^\top\right]^{-1} \boldsymbol{g}$$

Gauss–Newton

Example: square error

$\ell = \frac{1}{2} \|f - \boldsymbol{y}\|^2 \;\Rightarrow\; \frac{\partial^2 \ell}{\partial f^2} = \boldsymbol{I}$, so

$$\Delta \boldsymbol{\theta} = -\left[\frac{\partial f}{\partial \boldsymbol{\theta}} \frac{\partial f}{\partial \boldsymbol{\theta}}^\top\right]^{-1} \boldsymbol{g}$$

Example: log-likelihood (softmax)

$\ell = -\log \operatorname{softmax}(f)_{\boldsymbol{y}}$, with $\boldsymbol{p} = \operatorname{softmax}(f)$:

$$\frac{\partial^2 \ell}{\partial f^2} = \operatorname{diag}(\boldsymbol{p}) - \boldsymbol{p}\boldsymbol{p}^\top$$

Positive semi-definite and independent of the label $\boldsymbol{y}$: the Gauss–Newton matrix equals the **Fisher**, so GN = natural gradient (later).

First-order optimization: steepest descent

Take the Taylor expansion:

$$\mathcal{L}(\theta + \Delta\theta) = \mathcal{L}(\theta) + \boldsymbol{g}^\top \Delta\theta + \underbrace{\tfrac{1}{2}\Delta\theta^\top \boldsymbol{H}\, \Delta\theta + \cdots}_{\text{non-linear part}}$$

$\rightsquigarrow$ replace the non-linear part by a norm penalty $\frac{\lambda}{2} \|\Delta\theta\|^2$

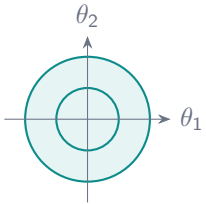
Gradient norm perspective

$$\mathcal{L}(\theta + \Delta\theta) \approx \mathcal{L}(\theta) + \boldsymbol{g}^\top \Delta\theta + \frac{\lambda}{2} \|\Delta\theta\|^2$$

First-order optimization: ℓ_2 steepest descent

Gradient norm perspective

$$\mathcal{L}(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}) + \boldsymbol{g}^\top \Delta\boldsymbol{\theta} + \frac{\lambda}{2} \underbrace{\|\Delta\boldsymbol{\theta}\|_2^2}_{\text{Euclidean norm}}$$



Euclidean balls

Minimize the RHS w.r.t. $\Delta\boldsymbol{\theta}$

$\Rightarrow$ differentiate and set derivative to zero

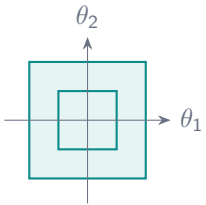
$\Rightarrow \boldsymbol{g} + \lambda\Delta\boldsymbol{\theta} = 0$

$$\Delta\boldsymbol{\theta} = -\tfrac{1}{\lambda}\,\boldsymbol{g}$$ **vanilla GD!**

First-order optimization: ℓ_∞ steepest descent

Gradient norm perspective

$$\mathcal{L}(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}) + \boldsymbol{g}^\top \Delta\boldsymbol{\theta} + \frac{\lambda}{2} \underbrace{\|\Delta\boldsymbol{\theta}\|_\infty^2}_{\text{infinity norm}}$$



infinity balls

Infinity norm

$\|\boldsymbol{v}\|_\infty = \max_i |v_i|$ the largest absolute entry; unit ball = hypercube

Minimizing the RHS w.r.t. $\Delta\boldsymbol{\theta}$ gives (derivation next):

$$\Delta\boldsymbol{\theta} = -\frac{\|\boldsymbol{g}\|_1}{\lambda} \operatorname{sign}(\boldsymbol{g})$$ **sign GD!**

Deriving ℓ_∞ steepest descent (to be skipped)

Split the step into a size and a direction: $\Delta\boldsymbol{\theta} = c\,\boldsymbol{t}$ with $c = \|\Delta\boldsymbol{\theta}\|_\infty \geq 0$ and $\|\boldsymbol{t}\|_\infty = 1$.
The objective becomes

$$c\,\boldsymbol{g}^\top \boldsymbol{t} + \frac{\lambda}{2} c^2.$$

Step 1: best direction

Coordinates decouple: each $t_i \in [-1, 1]$ minimizes $g_i t_i$ on its own, so $t_i = -\operatorname{sign}(g_i)$ and

$$\min_{\|\boldsymbol{t}\|_\infty \leq 1} \boldsymbol{g}^\top \boldsymbol{t} = -\sum_i |g_i| = -\|\boldsymbol{g}\|_1 .$$

Step 2: best size

Minimize $-c\,\|\boldsymbol{g}\|_1 + \frac{\lambda}{2} c^2$ over $c \geq 0$:

$$c^\star = \frac{\|\boldsymbol{g}\|_1}{\lambda}.$$

Combine: $\Delta\boldsymbol{\theta}^\star = c^\star\,\boldsymbol{t}^\star = -\frac{\|\boldsymbol{g}\|_1}{\lambda}\operatorname{sign}(\boldsymbol{g})$.

The ℓ_1 norm appears because it is the *dual* of the ℓ_∞ norm (more on this later).

From sign descent to Adam

Adam (Kingma & Ba, 2015)

$$\boldsymbol{m} \leftarrow \beta_1 \boldsymbol{m} + (1-\beta_1)\,\boldsymbol{g}$$

$$\boldsymbol{v} \leftarrow \beta_2 \boldsymbol{v} + (1-\beta_2)\,\boldsymbol{g}^2$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\hat{\boldsymbol{m}} \,/\, (\sqrt{\hat{\boldsymbol{v}}} + \epsilon)$$

$\hat{\boldsymbol{m}}$, $\hat{\boldsymbol{v}}$ bias-corrected; all operations element-wise

Equal betas: $\beta_1 = \beta_2 = \beta$

$\hat{m}_i$, $\hat{v}_i$ average g_i, g_i^2 over the gradient history with the *same* weights, so by Jensen $|\hat{m}_i| \le \sqrt{\hat{v}_i}$:

$$\frac{\hat{m}_i}{\sqrt{\hat{v}_i}} = \operatorname{sign}(\hat{m}_i) \cdot \rho_i, \qquad \rho_i = \frac{|\hat{m}_i|}{\sqrt{\hat{v}_i}} \in [0,1],$$

Takeaway

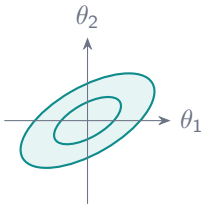
Adam with equal betas (in practice $\beta_1 = 0.9$ and $\beta_2 = 0.95$ when training large models) is **sign descent on the momentum**, damped coordinate-wise when the gradient keeps flipping sign.[1]

[1] Y. Zhou, T. Zhu, Y. Huo, S. P. Dash, C. Wang, S. Kaski, M. Sun. *The Hidden Ratio in Adam: Stable Structure, Compression, and Sign Dynamics.* NeurIPS 2026.

First-order optimization: Fisher-norm steepest descent

Gradient norm perspective

$$\mathcal{L}(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}) + \boldsymbol{g}^\top \Delta\boldsymbol{\theta} + \frac{\lambda}{2} \underbrace{\|\Delta\boldsymbol{\theta}\|^2_{\boldsymbol{F}_\theta}}_{\text{Fisher-induced norm}}, \qquad \|\Delta\boldsymbol{\theta}\|^2_{\boldsymbol{F}_\theta} = \Delta\boldsymbol{\theta}^\top \boldsymbol{F}_\theta \, \Delta\boldsymbol{\theta}$$



Fisher balls

Fisher information matrix:

$$\boldsymbol{F}_\theta = \mathbb{E}_{\boldsymbol{x},\, \hat{\boldsymbol{y}} \sim p_\theta(\cdot \mid \boldsymbol{x})} \Big[\nabla_\theta \log p_\theta(\hat{\boldsymbol{y}} \mid \boldsymbol{x}) \, \nabla_\theta \log p_\theta(\hat{\boldsymbol{y}} \mid \boldsymbol{x})^\top \Big]$$

Differentiate and set to zero: $\boldsymbol{g} + \lambda \boldsymbol{F}_\theta \Delta\boldsymbol{\theta} = 0$

$$\Delta\boldsymbol{\theta} = -\tfrac{1}{\lambda} \boldsymbol{F}_\theta^{-1} \boldsymbol{g}$$ **natural gradient!**

Why the Fisher norm?

Measure steps in probability space

To second order, the Fisher norm is the KL divergence between the model's prediction distributions before and after the step:

$$D_{\mathrm{KL}}\big(p_{\boldsymbol{\theta}} \,\|\, p_{\boldsymbol{\theta}+\Delta\boldsymbol{\theta}}\big) = \mathbb{E}_{p_{\boldsymbol{\theta}}}\left[\log \frac{p_{\boldsymbol{\theta}}(\hat{\boldsymbol{y}} \mid \boldsymbol{x})}{p_{\boldsymbol{\theta}+\Delta\boldsymbol{\theta}}(\hat{\boldsymbol{y}} \mid \boldsymbol{x})}\right] \approx \tfrac{1}{2}\, \Delta\boldsymbol{\theta}^{\top} \boldsymbol{F}_{\boldsymbol{\theta}}\, \Delta\boldsymbol{\theta}.$$

So the step size is judged by how much the *output distributions* change, not the raw parameters. Turns out to be a good idea in Deep Reinforcement Learning!

Upsides

- Invariant to smooth reparameterizations (to first order).
- For softmax cross-entropy (a log-likelihood loss), $\boldsymbol{F}_{\boldsymbol{\theta}}$ equals the Gauss–Newton matrix.

Downsides

- Like the Hessian, $\boldsymbol{F}_{\boldsymbol{\theta}}$ is $d \times d$: too expensive to store or invert.
- In practice it must be approximated: three ways next.

Estimating the natural gradient

With a mini-batch of n samples, stack the per-sample **scores** into a matrix:

$$\boldsymbol{S} = \begin{bmatrix} \boldsymbol{s}_1^\top \\ \vdots \\ \boldsymbol{s}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \qquad \boldsymbol{s}_i = \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\hat{\boldsymbol{y}}_i \mid \boldsymbol{x}_i), \qquad \hat{\boldsymbol{F}} = \tfrac{1}{n} \boldsymbol{S}^\top \boldsymbol{S}.$$

Important

$\hat{\boldsymbol{y}}_i \sim p_{\boldsymbol{\theta}}(\cdot \mid \boldsymbol{x}_i)$ must be **sampled from the model** (always hold in on-policy RL or log-softmax GN); Observed labels give the *empirical* Fisher, a different matrix.

We add damping $\gamma > 0$ and solve $\Delta\boldsymbol{\theta} = -\eta\,(\hat{\boldsymbol{F}} + \gamma \boldsymbol{I})^{-1} \boldsymbol{g}$.

1. **Woodbury**

   Matrix inversion in sample space $n \times n$, instead of $d \times d$

2. **Layer factorization**

   Block-diagonal over layers; each layer's scores factorize

3. **Diagonal Fisher**

   Keep only $\hat{F}_{jj}$: one number per parameter

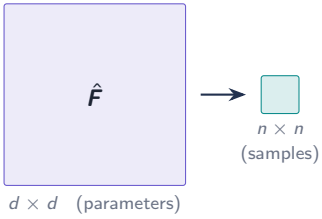
① Woodbury: parameter space → sample space

The damped sample Fisher is a low-rank update of $\gamma \boldsymbol{I}$, so by the **Woodbury identity**[1]:

$$\left(\gamma \boldsymbol{I}_d + \tfrac{1}{n}\boldsymbol{S}^\top \boldsymbol{S}\right)^{-1} = \frac{1}{\gamma}\Big[\boldsymbol{I}_d - \boldsymbol{S}^\top \big(\underbrace{n\gamma \boldsymbol{I}_n + \boldsymbol{S}\boldsymbol{S}^\top}_{n\times n}\big)^{-1}\boldsymbol{S}\Big]$$

$$\Delta\boldsymbol{\theta} = -\frac{\eta}{\gamma}\Big[\boldsymbol{g} - \boldsymbol{S}^\top \big(n\gamma \boldsymbol{I}_n + \boldsymbol{S}\boldsymbol{S}^\top\big)^{-1}\boldsymbol{S}\boldsymbol{g}\Big] \qquad \text{NG}$$

Only the $n \times n$ **Gram matrix** $\boldsymbol{S}\boldsymbol{S}^\top$ (inner products between samples' scores) is ever inverted.



Cost

$O(n^2 d + n^3)$ instead of $O(d^3)$.
A win whenever $n \ll d$, which is the usual case for neural nets.

[1] M. Sun. *Randomized Advantage Transformation (RAT): Computing Natural Policy Gradients via Direct Backpropagation.* ICML 2026.

② Factorization across layers

Step 1: block-diagonal over layers

Ignore cross-layer blocks, $\hat{\boldsymbol{F}} \approx \operatorname{diag}(\hat{\boldsymbol{F}}_1, \ldots, \hat{\boldsymbol{F}}_L)$, and solve each layer separately.

Notation: $\otimes$ is the Kronecker product, $\boldsymbol{P} \otimes \boldsymbol{Q} = [\, P_{ij}\boldsymbol{Q} \,]$; e.g. $\boldsymbol{h} \otimes \boldsymbol{\delta} = [\, h_1\boldsymbol{\delta};\ h_2\boldsymbol{\delta};\ \ldots \,]$.

Step 2: each score factorizes

As in the backward pass, $\nabla_{\boldsymbol{W}} \log p_{\boldsymbol{\theta}}(\hat{\boldsymbol{y}}_i \mid \boldsymbol{x}_i) = \boldsymbol{\delta}_i \boldsymbol{h}_i^\top$, so $\boldsymbol{s}_i = \operatorname{vec}(\boldsymbol{\delta}_i \boldsymbol{h}_i^\top) = \boldsymbol{h}_i \otimes \boldsymbol{\delta}_i$.

vec stacks a matrix's columns. Samples $\boldsymbol{h}_i$, $\boldsymbol{\delta}_i$ are the rows of $\boldsymbol{A} \in \mathbb{R}^{n \times d_{\text{in}}}$, $\boldsymbol{D} \in \mathbb{R}^{n \times d_{\text{out}}}$.

Parameter space: K-FAC (approximate)

$$\hat{\boldsymbol{F}}_l = \tfrac{1}{n} \textstyle\sum_i (\boldsymbol{h}_i \boldsymbol{h}_i^\top) \otimes (\boldsymbol{\delta}_i \boldsymbol{\delta}_i^\top) \approx \left(\tfrac{1}{n} \boldsymbol{A}^\top \boldsymbol{A}\right) \otimes \left(\tfrac{1}{n} \boldsymbol{D}^\top \boldsymbol{D}\right)$$

$$\Delta \boldsymbol{W} = -\eta \left(\tfrac{1}{n} \boldsymbol{D}^\top \boldsymbol{D}\right)^{-1} \boldsymbol{G} \left(\tfrac{1}{n} \boldsymbol{A}^\top \boldsymbol{A}\right)^{-1}$$

Invert $d_{\text{out}} \times d_{\text{out}}$ and $d_{\text{in}} \times d_{\text{in}}$.

Sample space: Gram matrix (exact)

$$\begin{aligned} \left(\boldsymbol{S}_l \boldsymbol{S}_l^\top\right)_{ij} &= \operatorname{vec}(\boldsymbol{\delta}_i \boldsymbol{h}_i^\top)^\top \operatorname{vec}(\boldsymbol{\delta}_j \boldsymbol{h}_j^\top) \\ &= (\boldsymbol{h}_i \otimes \boldsymbol{\delta}_i)^\top (\boldsymbol{h}_j \otimes \boldsymbol{\delta}_j) = (\boldsymbol{h}_i^\top \boldsymbol{h}_j)(\boldsymbol{\delta}_i^\top \boldsymbol{\delta}_j) \\ \Rightarrow\ \boldsymbol{S}_l \boldsymbol{S}_l^\top &= (\boldsymbol{A}\boldsymbol{A}^\top) \odot (\boldsymbol{D}\boldsymbol{D}^\top) \end{aligned}$$

By Woodbury: only an $n \times n$ solve.

③ Diagonal Fisher

Keep only the diagonal, one curvature estimate per parameter: $\hat{F}_{jj} = \frac{1}{n}\sum_i s_{ij}^2$.

$$\Delta\theta_j = -\eta \, \frac{g_j}{\hat{F}_{jj} + \gamma}$$

diagonal natural gradient

Sophia (Liu, Li, Hall, Liang & Ma, ICLR 2024)

$$\boldsymbol{m} \leftarrow \beta_1 \boldsymbol{m} + (1-\beta_1)\,\boldsymbol{g}$$

$$\boldsymbol{h} \leftarrow \beta_2 \boldsymbol{h} + (1-\beta_2)\,\hat{\boldsymbol{h}} \quad \text{every } k = 10 \text{ steps}$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \mathrm{clip}\Big(\frac{\boldsymbol{m}}{\max(\gamma \boldsymbol{h},\, \epsilon)},\, 1\Big)$$

Curvature $\hat{\boldsymbol{h}}$: Hutchinson $\boldsymbol{u} \odot (\boldsymbol{H}\boldsymbol{u})$, $\boldsymbol{u} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$; or Gauss–Newton $\hat{\boldsymbol{g}} \odot \hat{\boldsymbol{g}}$ with $\hat{\boldsymbol{y}}$ sampled from the model.

Reading / caveats

$\boldsymbol{m}/\boldsymbol{h}$ is a per-coordinate **Newton step**. Clipping caps each step at η; where curvature is tiny or negative it saturates to η sign($\boldsymbol{m}$): **sign descent**.

Reported 2× speedup over Adam (GPT, 125M–1.5B); with a re-tuned AdamW such claims largely vanish (Wen et al., 2026).

Natural gradient (Gauss-Newton) estimators at a glance

Estimator	Inverts	Cost	Keeps
Exact $\hat{\boldsymbol{F}}$	$d \times d$	$O(d^3)$	everything
① Woodbury	$n \times n$	$O(n^2 d + n^3)$	all of the sampled $\hat{\boldsymbol{F}}$
② Layer Gram	$n \times n$ per layer	$O\big(n^2(d_{\text{in}}+d_{\text{out}}) + n^3\big)$ / layer	within-layer blocks
② K-FAC	$d_{\text{in}} \times d_{\text{in}}$, $d_{\text{out}} \times d_{\text{out}}$ per layer	$O(d_{\text{in}}^3 + d_{\text{out}}^3)$ / layer	Kronecker approximation
③ Diagonal	nothing	$O(nd)$	per-parameter scale

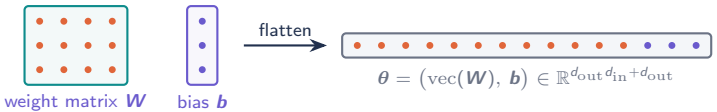
Recap: classical methods at a glance

Method	Model of the non-linear part	Update $\Delta\theta$
Newton	$\frac{\lambda}{2}\,\Delta\boldsymbol{\theta}^\top \boldsymbol{H}\,\Delta\boldsymbol{\theta}$	$-\boldsymbol{H}^{-1}\boldsymbol{g}$
Gauss–Newton	drop curvature of the model	$-\big[\frac{\partial f}{\partial \boldsymbol{\theta}}\frac{\partial f}{\partial \boldsymbol{\theta}}^\top\big]^{-1}\boldsymbol{g}$
ℓ_2 steepest descent	$\frac{\lambda}{2}\,\lVert\Delta\boldsymbol{\theta}\rVert_2^2$	$-\frac{1}{\lambda}\,\boldsymbol{g}$
ℓ_∞ steepest descent	$\frac{\lambda}{2}\,\lVert\Delta\boldsymbol{\theta}\rVert_\infty^2$	$-\frac{\lVert\boldsymbol{g}\rVert_1}{\lambda}\,\mathrm{sign}(\boldsymbol{g})$
$\hookrightarrow$ Adam, $\beta_1{=}\beta_2{=}0$	same direction, fixed step	$-\eta\,\mathrm{sign}(\boldsymbol{g})$
Natural gradient	$\frac{\lambda}{2}\,\lVert\Delta\boldsymbol{\theta}\rVert_{\boldsymbol{F}_\theta}^2$	$-\frac{1}{\lambda}\boldsymbol{F}_\theta^{-1}\boldsymbol{g}$
Sophia	diagonal Hessian, clipped	$-\eta\,\mathrm{clip}\big(\boldsymbol{m}/\max(\gamma\boldsymbol{h},\epsilon),1\big)$

Matrix Optimizers: Muon, Shampoo and SOAP

Starting point: a linear layer

$$\boldsymbol{z} = \boldsymbol{W}\boldsymbol{h} + \boldsymbol{b}, \qquad \boldsymbol{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \quad \boldsymbol{b} \in \mathbb{R}^{d_{\text{out}}}$$



Gradient descent's view

All parameters become one long vector, measured in the ℓ_2 norm:

$$\|\Delta\boldsymbol{\theta}\|_2^2 = \|\Delta\boldsymbol{W}\|_F^2 + \|\Delta\boldsymbol{b}\|_2^2 .$$

Every entry of $\boldsymbol{W}$ is an independent coordinate.

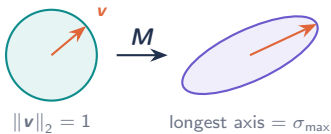
Matrix optimizer's view

$\boldsymbol{W}$ is not just a list of numbers: it is a **linear map** $\boldsymbol{x} \mapsto \boldsymbol{W}\boldsymbol{x}$. So keep $\boldsymbol{W}$ as a matrix and measure $\Delta\boldsymbol{W}$ with a norm that is *defined for matrices*. The bias $\boldsymbol{b}$ is just a vector and is handled separately.

A norm made for matrices: the spectral norm

What we care about is how much an update can change the layer's *output*:

$$\|\Delta \boldsymbol{W}\boldsymbol{h}\|_2 \;\leq\; \|\Delta \boldsymbol{W}\|_* \, \|\boldsymbol{h}\|_2 \,.$$



Spectral norm

$$\|\boldsymbol{M}\|_* = \max_{\boldsymbol{v}\neq 0} \frac{\|\boldsymbol{M}\boldsymbol{v}\|_2}{\|\boldsymbol{v}\|_2} = \sigma_{\max}(\boldsymbol{M})$$

Why not Frobenius?

$\|\boldsymbol{M}\|_* \leq \|\boldsymbol{M}\|_F \leq \sqrt{\operatorname{rank} \boldsymbol{M}}\, \|\boldsymbol{M}\|_*$, so the flattened ℓ_2 size can overstate the change in the output by up to $\sqrt{\operatorname{rank}}$.

Spectral steepest descent

Steepest descent on $\boldsymbol{W}$ with gradient $\boldsymbol{G} = \nabla_{\boldsymbol{W}} \mathcal{L}$, measured in the **spectral norm**:

Gradient norm perspective

$$\mathcal{L}(\boldsymbol{W} + \Delta \boldsymbol{W}) \approx \mathcal{L}(\boldsymbol{W}) + \langle \boldsymbol{G}, \Delta \boldsymbol{W} \rangle + \frac{\lambda}{2} \underbrace{\|\Delta \boldsymbol{W}\|_*^2}_{\text{spectral norm}}$$

The dual of the spectral norm is the **nuclear norm** $\|\boldsymbol{G}\|_{\text{nuc}} = \sum_i \sigma_i$, and with the SVD $\boldsymbol{G} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$, $\arg\max_{\|\boldsymbol{T}\|_* = 1} \langle \boldsymbol{G}, \boldsymbol{T} \rangle = \boldsymbol{U}\boldsymbol{V}^\top$, so

$$\Delta \boldsymbol{W} = -\frac{\|\boldsymbol{G}\|_{\text{nuc}}}{\lambda} \boldsymbol{U}\boldsymbol{V}^\top$$

orthogonalized gradient

Matrix sign

$\boldsymbol{U}\boldsymbol{V}^\top$ is the **matrix sign** of $\boldsymbol{G}$: $\operatorname{msgn}(\boldsymbol{G}) = \boldsymbol{U}\boldsymbol{V}^\top$. Every singular value is set to 1, the matrix analogue of $\operatorname{sign}(\boldsymbol{g})$. (Equivalently $\boldsymbol{G}(\boldsymbol{G}^\top \boldsymbol{G})^{-1/2}$ for full column rank.)

Deriving the orthogonalized gradient (To be skipped)

Split the step into a size and a direction: $\Delta \boldsymbol{W} = c\, \boldsymbol{T}$ with $c = \|\Delta \boldsymbol{W}\|_* \geq 0$ and $\|\boldsymbol{T}\|_* = 1$. The objective becomes

$$c\,\langle \boldsymbol{G}, \boldsymbol{T} \rangle + \frac{\lambda}{2} c^2, \qquad \langle \boldsymbol{G}, \boldsymbol{T} \rangle = \operatorname{tr}(\boldsymbol{G}^\top \boldsymbol{T}).$$

Step 1: best direction

With $\boldsymbol{G} = \sum_i \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^\top$:

$$\langle \boldsymbol{G}, \boldsymbol{T} \rangle = \sum_i \sigma_i\, \boldsymbol{u}_i^\top \boldsymbol{T} \boldsymbol{v}_i \;\geq\; -\sum_i \sigma_i = -\|\boldsymbol{G}\|_{\mathrm{nuc}},$$

since $|\boldsymbol{u}_i^\top \boldsymbol{T} \boldsymbol{v}_i| \leq \|\boldsymbol{T}\|_* = 1$ for unit $\boldsymbol{u}_i, \boldsymbol{v}_i$. Equality holds for $\boldsymbol{T}^\star = -\boldsymbol{U}\boldsymbol{V}^\top$.

Step 2: best size

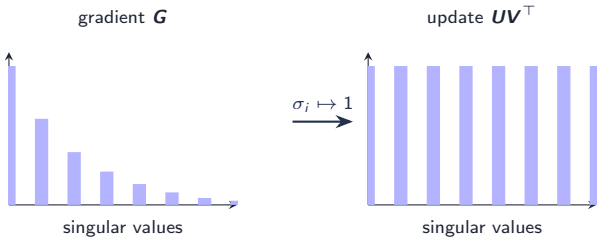
Minimize $-c\,\|\boldsymbol{G}\|_{\mathrm{nuc}} + \frac{\lambda}{2} c^2$ over $c \geq 0$:

$$c^\star = \frac{\|\boldsymbol{G}\|_{\mathrm{nuc}}}{\lambda}.$$

Combine: $\Delta \boldsymbol{W}^\star = -\frac{\|\boldsymbol{G}\|_{\mathrm{nuc}}}{\lambda}\, \boldsymbol{U}\boldsymbol{V}^\top$.

Same recipe as $\ell_\infty \to$ sign descent: the nuclear norm appears as the *dual* of the spectral norm.

What orthogonalization does



Intuition

Keep the gradient's singular *directions*, but give every direction the *same* step size. Rare directions with small σ_i are no longer drowned out.

Width-independent step size

With $\|v\|_{\mathrm{RMS}} = \|v\|_2 / \sqrt{d}$, the induced norm is $\|M\|_{\mathrm{RMS}\to\mathrm{RMS}} = \sqrt{d_{\mathrm{in}}/d_{\mathrm{out}}}\ \|M\|_*$. Scaling $UV^\top$ by $\eta\sqrt{d_{\mathrm{out}}/d_{\mathrm{in}}}$ gives $\|\Delta W\|_{\mathrm{RMS}\to\mathrm{RMS}} = \eta$ for every layer, regardless of width.

Muon: MomentUm Orthogonalized by Newton–Schulz

Muon update for a hidden matrix W

1. momentum: $\boldsymbol{B} \leftarrow \beta \boldsymbol{B} + \boldsymbol{G}$
2. orthogonalize: $\boldsymbol{O} \leftarrow \mathrm{NS}_5(\boldsymbol{B}) \approx \boldsymbol{U}\boldsymbol{V}^\top$
3. step: $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\, \boldsymbol{O}$

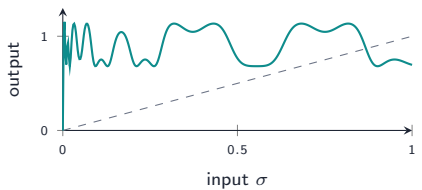
Computing an SVD every step is too slow, so Muon *approximates* $\boldsymbol{U}\boldsymbol{V}^\top$ with a few matrix multiplications. The bias $\boldsymbol{b}$ uses a standard optimizer such as AdamW.

Newton–Schulz iteration

$\boldsymbol{X}_0 = \boldsymbol{B} / \|\boldsymbol{B}\|_F$, then repeat 5 times:

$$\boldsymbol{X} \leftarrow c_1 \boldsymbol{X} + c_3 (\boldsymbol{X}\boldsymbol{X}^\top)\boldsymbol{X} + c_5 (\boldsymbol{X}\boldsymbol{X}^\top)^2 \boldsymbol{X}$$

$(c_1, c_3, c_5) = (3.4445,\ -4.7750,\ 2.0315)$



acts on each singular value: 5 steps push every σ into $\approx [0.7, 1.2]$

Muon variants (I): stability and adaptivity

MuonClip (Kimi Team, 2025)

Problem: at scale, attention logits $\boldsymbol{q}_i^\top \boldsymbol{k}_j$ blow up.

Fix: after each step, per head h,

$$\gamma_h = \min\left(1, \frac{\tau}{S^h_{\max}}\right)$$

and scale $\boldsymbol{W}_q^h, \boldsymbol{W}_k^h$ by $\sqrt{\gamma_h}$.

Result: Kimi K2 (1T MoE) trained on 15.5T tokens with zero loss spikes.

NorMuon (Li et al., ICML 2026)

Problem: orthogonalized updates give neurons very uneven norms.

Fix: orthogonalize, *then* normalize rows by a per-neuron second moment:

$$\mathrm{NS}(\boldsymbol{B}) \to \text{row-normalize}$$

Result: at 1.1B, 21.7% more efficient than Adam and 11.3% than Muon.

Muon^2 (Liu et al., EMNLP 2026)

Problem: ill-conditioned momentum slows Newton–Schulz.

Fix: Adam-style second-moment preconditioning *before* orthogonalizing:

$$\boldsymbol{B} \oslash \sqrt{\boldsymbol{V}} \to \mathrm{NS}(\cdot)$$

Result: up to 13B, 40% fewer NS iterations and up to 1/4 less training time.

NorMuon and Muon^2: orthogonalization and element-wise adaptivity are complementary.

Muon variants (II): curvature and efficiency

Newton–Muon (Du & Su, 2026)

Idea: put input curvature back in, with $\boldsymbol{A}$ the stacked layer inputs:

$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta\, \mathrm{msgn}\big(\boldsymbol{G}(\boldsymbol{A}^\top \boldsymbol{A})^{-1}\big)$$

Muon is the case $\boldsymbol{A}^\top \boldsymbol{A} \propto \boldsymbol{I}$; this is K-FAC's input factor.

Result: GPT-2 speedrun: 6% fewer steps, about 4% less wall-clock.

Polar Express (Amsel et al., ICLR 2026)

Idea: replace the fixed Newton–Schulz polynomial with an optimal (minimax) polynomial at *each* iteration.

Still matmul-only, fast early and asymptotically, and stable in bfloat16.

Result: lower GPT-2 validation loss inside Muon across learning rates.

Dion (Ahn et al., 2025)

Idea: replace Newton–Schulz with amortized power iteration, orthonormalizing a rank-r subspace; error feedback keeps what low rank drops.

Works on sharded weights (DDP, FSDP, TP).

Result: stays practical at Llama-3 405B scale.

Shampoo, Muon and SOAP

Shampoo (Gupta, Koren & Singer, 2018)

$\boldsymbol{L} \leftarrow \boldsymbol{L} + \boldsymbol{G}\boldsymbol{G}^\top, \quad \boldsymbol{R} \leftarrow \boldsymbol{R} + \boldsymbol{G}^\top \boldsymbol{G},$
$\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta\, \boldsymbol{L}^{-1/4} \boldsymbol{G} \boldsymbol{R}^{-1/4}$

Why: full-matrix AdaGrad uses $\boldsymbol{C}^{-1/2}\boldsymbol{g}$, $\boldsymbol{C} = \sum \boldsymbol{g}\boldsymbol{g}^\top$ $(mn \times mn)$. Since $\boldsymbol{C} \preceq r\,(\boldsymbol{L}^{1/2} \otimes \boldsymbol{R}^{1/2})$, $\boldsymbol{C}^{-1/2} \approx \boldsymbol{L}^{-1/4} \otimes \boldsymbol{R}^{-1/4}$: memory $m^2 + n^2$, not $(mn)^2$.

Muon is Shampoo without memory

Drop the accumulation: $\boldsymbol{L} = \boldsymbol{G}\boldsymbol{G}^\top$, $\boldsymbol{R} = \boldsymbol{G}^\top \boldsymbol{G}$. With $\boldsymbol{G} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$:

$$\boldsymbol{L}^{-1/4} \boldsymbol{G} \boldsymbol{R}^{-1/4} = \boldsymbol{U}\boldsymbol{\Sigma}^{-1/2}\,\boldsymbol{\Sigma}\,\boldsymbol{\Sigma}^{-1/2}\boldsymbol{V}^\top = \boldsymbol{U}\boldsymbol{V}^\top$$

Exactly Muon's direction, with only momentum as state.

SOAP: Adam in Shampoo's eigenbasis (Vyas et al., ICLR 2025)

Every f steps take the eigenvectors $\boldsymbol{Q}_L, \boldsymbol{Q}_R$ of $\boldsymbol{L}, \boldsymbol{R}$; rotate $\boldsymbol{G}' = \boldsymbol{Q}_L^\top \boldsymbol{G} \boldsymbol{Q}_R$, run an **Adam** step on $\boldsymbol{G}'$, rotate back. Shampoo is Adafactor in this basis; SOAP uses Adam instead, adding only one hyperparameter (f). **Result:** over 40% fewer steps than AdamW at 360M/660M, and holds up at 30B–72B (Khona et al., 2026).

Matrix optimizers at a glance

	Muon	**Shampoo**	**SOAP**
Update direction	$\boldsymbol{UV}^\top$ of momentum	$\boldsymbol{L}^{-1/4}\boldsymbol{G}\boldsymbol{R}^{-1/4}$	Adam in eigenbasis of $\boldsymbol{L}, \boldsymbol{R}$
Extra state	mn	$m^2 + n^2$	$m^2 + n^2 + 2mn$
Expensive step	Newton–Schulz, every step	inverse roots, every k steps	eigenvectors, every f steps
Extra knobs	few	many (grafting, ϵ, k, ...)	one (f)

Common thread

All three treat $\boldsymbol{W}$ as a **matrix** and precondition with matrices, not element-wise scalars. In a careful benchmark of LLM pretraining, the fastest optimizers were all of this kind (Wen et al., 2026).

Future research directions

1. Across training stages

Optimizers that work from pretraining through fine-tuning to RL post-training, without the mismatch of switching optimizers between stages.

2. Theory: which norm?

Why does orthogonalization help? Which norm should each layer use, and what can we prove about convergence?

3. Cheaper curvature

Connect sample-space (Gram, Woodbury) and parameter-space (K-FAC, Shampoo) factorizations with orthogonalized updates.

4. Optimizing RL

Non-stationary, policy-dependent data: scalable natural policy gradients and matrix optimizers for RL post-training.

5. Stability at scale

Principled control of weight and logit growth (cf. MuonClip), and its interplay with weight decay and schedules.

6. Systems co-design

Distributed and low-precision orthogonalization, communication-efficient updates, smaller optimizer state.

Theory $\leftrightarrow$ practice: each direction pairs a modelling question with an engineering one.

References

Natural Gradient Works Efficiently in Learning. Amari (1998).

Optimizing Neural Networks with Kronecker-factored Approximate Curvature. Martens & Grosse (2015).

Efficient Subsampled Gauss-Newton and Natural Gradient Methods for Training Neural Networks. Ren & Goldfarb (2019).

Randomized Advantage Transformation (RAT): Computing Natural Policy Gradients via Direct Backpropagation. Sun (2026), ICML.

Adam: A Method for Stochastic Optimization. Kingma & Ba (2015).

Dissecting Adam: The Sign, Magnitude and Variance of Stochastic Gradients. Balles & Hennig (2018).

The Hidden Ratio in Adam: Stable Structure, Compression, and Sign Dynamics. Zhou, Zhu, Huo, Dash, Wang, Kaski & Sun (2026), NeurIPS.

Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. Liu, Li, Hall, Liang & Ma (2024), ICLR.

Universal Majorization-Minimization Algorithms. Streeter (2023).

Feature Learning in Infinite-Width Neural Networks. Yang & Hu (2020).

Infinite Limits of Multi-Head Transformer Dynamics. Bordelon, Chaudhry & Pehlevan (2024).

Muon: An Optimizer for Hidden Layers in Neural Networks. Jordan et al. (2024), blog post.

Old Optimizer, New Norm: An Anthology. Bernstein & Newhouse (2024).

Shampoo: Preconditioned Stochastic Tensor Optimization. Gupta, Koren & Singer (2018).

SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. Vyas et al. (2025), ICLR.

Kimi K2: Open Agentic Intelligence. Kimi Team (2025), arXiv:2507.20534.

NorMuon: Making Muon More Efficient and Scalable. Li, Liu, Liang, Chen & Zhao (2026), ICML.

Muon^2: Boosting Muon via Adaptive Second-Moment Preconditioning. Liu et al. (2026), EMNLP.

The Newton–Muon Optimizer. Du & Su (2026), arXiv:2604.01472.

The Polar Express: Optimal Matrix Sign Methods and Their Application to the Muon Algorithm. Amsel, Persson, Musco & Gower (2026), ICLR.

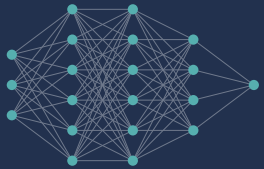
Dion: Distributed Orthonormalized Updates. Ahn et al. (2025), arXiv:2504.05295.

SOAP, Muon, and Beyond: Pushing LLM Pretraining Scales. Khona et al. (2026).

Fantastic Pretraining Optimizers and Where to Find Them. Wen et al. (2026), ICLR.

Thank you!

Questions & discussion



Mingfei Sun

The University of Manchester

mingfei.sun@manchester.ac.uk

mingfeisun.github.io

Parts adapted from MIT 6.7960 Lecture 7 (J. Bernstein), CC BY-NC-SA.